Arrays - VIII

Thursday, 7 May 2026 6:41 AM

arr = {1, 2, 3, 4, 5, 6}

Sub-array = continuous part of an array.

arr = {1, 2, 3, 4, 5, 6}

~~{2, 5, 6}~~

Subarray -

{1,2,3,4,5,6} {2,3,4,5,6} {3,4,5,6} {4,5,6} {5,6} {6}

{1,2,3,4,5} {2,3,4,5} {3,4,5} {4,5} {5}

{1,2,3,4} {2,3,4} {3,4} {4}

{1,2,3} {2,3} {3}

{1,2} {2}

{1}

arr = {1, 2, 3, 4, 5, 6}

Sub-sequence - part of an array with same relative order.

{2, 5, 6}

{2, 4, 6}

{3, 4}

~~{3, 5, 2}~~

{2, 3, 5}

arr = {1, 2, 3, 4, 5, 6}

{1, 3, 5}

{2, 4, 6}

{1, 3}

{2, 4}

{3, 5}

{3, 4, 5}

subarray → subsequence

Q.1) V.V.V.V.V.V.V.V.V.V.IMP

arr = {1, 2, -4, 3, 6, 8, -14, -3, -2}

17

{3, 6, 8}

Maximum subarray sum

max

subarray

{1, 2, -4, 3, 6, 8, -14, -3, -2} = -3

{1, 2, -4, 3, 6, 8, -14, -3} = -1

{1, 2, -4, 3, 6, 8, -14} = 2

{1, 2, -4, 3, 6, 8} = 16

{1, 2, -4, 3, 6} = 8

{1, 2, -4, 3} = 2

{1, 2, -4} = -1

{1, 2} = 3

{1} = 1

16

{2, -4, 3, 6, 8, -14, -3, -2} = -4

{2, -4, 3, 6, 8, -14, -3} = -2

{2, -4, 3, 6, 8, -14} = 1

{2, -4, 3, 6, 8} = 15

{2, -4, 3, 6} = 7

{2, -4, 3} = 1

{2, -4} = -2

{2} = 2

arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4}

{-2}

sum = 0

max → ans

{-2} = sum

{-2, 1} sum

{-2, 1, -3} sum

{-2, 1, -3, 4, -1, 2} sum = -2 + 1 + -3 + 4 - 1 + 2

sum

sum

S

S

S

Approach :-

1) Subarray find karo.

2) sum : ghum ghumake sum calculate karo

3) sum → max nikalo

Code -

```
int max = Integer.MIN_VALUE;
for (int i=0; i<arr.length; i++) {
    for (int j=i; j<arr.length; j++) {
        int sum = 0;
        for (int k=i; k<=j; k++) {
            sum += arr[k];
        }
        max = Math.max(max, sum);
    }
}
```

i → subarray start index

j → subarray end index

// TC - O(n^3)

// SC - O(1)

S.I

S.P

Approach 2-

Code -

```
int max = Integer.MIN_VALUE;
for (int i=0; i<arr.length; i++) {
    int sum = 0;
    for (int j=i; j<arr.length; j++) {
        sum += arr[j];
        max = Math.max(max, sum);
    }
}
```

// TC - O(n^2)

// SC - O(1)

Approach 3 - Kadane's Algorithm

positive number + ~~negative number~~

= ~~chhota number~~

push

+10

⇒ Index

extend

new subarray.

Steps -

1) current element → currentSum

2) maxSum update.

3) If currentSum < 0

currSum = 0;

Dry Run -

arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4}

element	Current Sum	Max Sum
-2	-2 → 0	-2
1	1	1
-3	-2 → 0	1
4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

MAX

6

{-2, -1}

element	currentSum	MaxSum
-2	-2 → 0	-2
-1	-1 → 0	-1

-1

Code -

```
int currentSum = 0;
int maxSum = Integer.MIN_VALUE;
for (int i=0; i<arr.length; i++) {
    currentSum += arr[i];
    maxSum = Math.max(maxSum, currentSum);
    if (currentSum < 0) {
        currentSum = 0;
    }
}
print (maxSum);
}
```

// TC - O(n)

// SC - O(1)

Q.2) Print ~~sub~~ maximum subarray sum

{1, -4, 2, -3, 8, 4, 11, -14}

{8, 4, 11}

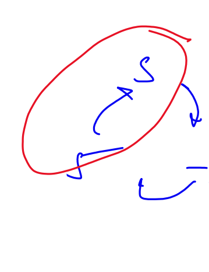
Q.3) Maximum Product Subarray.

{2, 3, -2, 4}

6

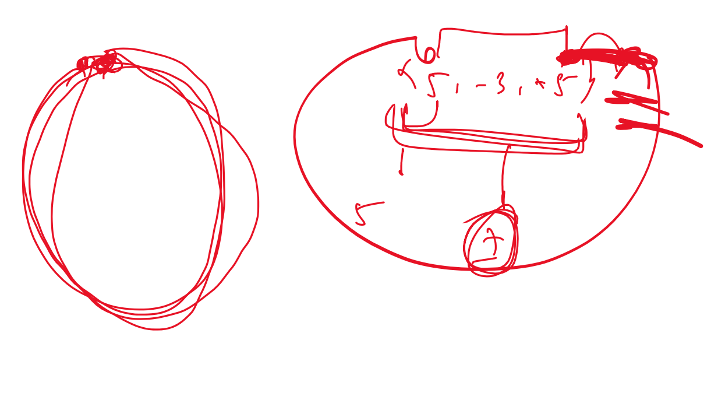
Trick - negative × negative = positive

Q.4) Circular subarray sum.



{5} -3 {5}

{5, 5}

10